Interface Reliability in Verified Software-Agent Evaluation

Separating Output, Patch, and Semantic Failures in Long-Horizon Agent Benchmarks

Working Paper - May 2026

Author: -pk

Pilot methodology study based on a 10-task software-agent challenge set and five replication runs

Abstract

Software-agent benchmarks often report whether an agent solves a task, but a single failure label can hide very different causes: invalid output, schema errors, malformed patches, non-applying edits, verifier failures, or genuine semantic mistakes. We study this problem in a 10-task pilot benchmark for software-repair agents. In an initial raw-diff setup, many failures were evaluator-unreachable artifacts caused by backend failures, missing final patches, and malformed diffs. We then replaced raw diff generation with structured JSON edits, deterministic patch generation, and schema repair. Across five replications of the repaired JSON interface, semantic correctness conditional on patch application remained high for all prompt conditions, while variation in total solve rate was driven mainly by patch applicability. These results show that interface reliability can dominate measured agent performance and that software-agent benchmarks should report layered metrics: output validity, schema usability, patch applicability, verifier success, and false completion. We present Verified Experience Trajectories as a motivating framework for future work, but our empirical contribution is narrower: before evaluating whether verified-experience context or training improves agents, benchmarks must make semantic success reachable and distinguish interface failure from task failure.

Keywords: software agents; benchmark methodology; interface reliability; structured outputs; patch evaluation; failure decomposition; verified experience

1. Introduction

Long-horizon software agents are increasingly evaluated as systems that change an environment state rather than merely produce an answer. A coding agent succeeds when it modifies a repository and passes verification, not when it describes a plausible fix. This shift is visible in benchmarks such as WebArena, OSWorld, SWE-bench Verified, and tau-bench, which evaluate agents through interaction, state change, executable task completion, or final database state rather than ordinary question answering [1-4].

State-changing evaluation introduces a methodological problem: a task can fail for many reasons unrelated to semantic problem solving. A model may time out, fail to emit a final artifact, output invalid JSON, produce a malformed diff, generate a patch that cannot be applied, or apply a patch that fails tests. Collapsing these into a single failure label can mislead readers about whether the agent failed at reasoning, tool use, formatting, schema compliance, or evaluator reachability.

This paper argues that interface reliability is a prerequisite for verified agent evaluation. The broad research motivation is the Verified Experience Hypothesis: future agents may learn from trajectories that include goals, states, actions, observations, errors, verifications, and memory updates. The empirical contribution here is earlier and narrower: before testing whether verified-experience training helps, benchmarks must make semantic success reachable and report where failures occur.

Contributions

- We define a layered failure decomposition for software-agent evaluation: raw output validity, schema usability, patch generation, patch applicability, verified solve, and false completion.
- We introduce a pilot challenge benchmark and compare raw diff generation with structured JSON edits plus deterministic patch generation and schema repair.
- We report a five-replication stability study showing that solve-if-applied remains high across prompt conditions, while total solve rate is driven mostly by patch applicability.
- We show why prompt-condition rankings should be treated cautiously when interface failures dominate outcome variance.
- We position Verified Experience Trajectories as future work rather than as a result proven by this pilot.

2. Background and Motivation

2.1 From answer generation to state transformation

Modern agent benchmarks increasingly judge whether systems transform an environment into a verified target state. WebArena evaluates realistic web tasks; OSWorld evaluates multimodal agents in real computer environments; SWE-bench Verified evaluates software issue resolution; and tau-bench evaluates tool-using agents by final database state [1-4]. These benchmarks push beyond ordinary answer generation, but they also make evaluator reachability a central issue.

2.2 Tool use, memory, and feedback motivate trajectory-level evaluation

ReAct interleaves reasoning and acting, Toolformer studies learned API calls, Reflexion stores feedback as episodic memory, and MemGPT treats memory management as an operating-system-like problem [5-8]. These lines of work motivate trajectory-level evaluation, but they do not remove the need to distinguish interface failure from semantic task failure.

2.3 Verified Experience Trajectories as motivation

A Verified Experience Trajectory (VET) records goal, initial state, constraints, available tools, actions, observations, errors, corrections, verifier results, final state, human feedback, and memory updates. VETs

are useful as a conceptual target because they preserve more information than prompt-answer pairs or final patches. This paper does not test VET training or scaling. It tests a prerequisite: whether the benchmark interface permits semantic task success to be measured reliably.

```
VET = (goal, initial_state, constraints, tools, actions, observations,
    errors, corrections, verifier_results, final_state, memory_updates)
```

3. Failure Decomposition for Verified Agent Evaluation

The central methodological claim is that software-agent failures should be decomposed before they are interpreted. A malformed diff, invalid JSON object, non-applying patch, verifier-failing patch, and false success claim are different events. Only some are evidence of semantic problem-solving failure.

Table 1. Layered failure decomposition for software-agent evaluation.

Layer	Question answered	Example failure
Raw output exists	Did the model emit an artifact?	Timeout, backend drop, no final patch
Valid JSON	Is the output parseable?	invalid_json
Schema usable	Does it match the edit schema?	edits is not a list
Patch generated	Can edits become a candidate patch?	missing file path, forbidden test edit
Patch applied	Does the patch apply cleanly?	context mismatch, path mismatch
Verified solve	Do tests/verifiers pass?	semantic bug remains
False completion	Did it claim success despite failure?	patch applied but tests failed

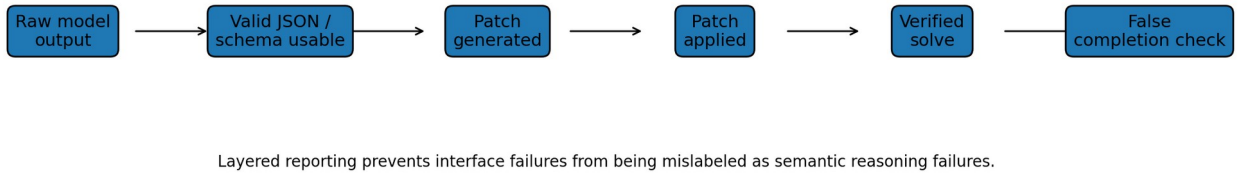

Figure 1. Layered evaluation pipeline used in the repaired JSON interface.

4. Pilot Benchmark and Methods

4.1 Challenge task set

The pilot uses 10 synthetic but non-trivial Python software-repair tasks. Each task includes task.md, issue text, metadata, repository files, tests, and a gold patch. Gold controls solve all tasks, and empty-patch controls solve none. The task set is deliberately small and should be interpreted as a pilot rather than as an external benchmark.

Table 2. Challenge task inventory.

Task	Bug type	What it tests
001	edge_case	reject leading/trailing whitespace in username normalization
002	multi_file	consistent env parsing across parser and settings modules
003	misleading_obvious_fix	truncation logic with ellipsis and tiny limits
004	state_mutation	nested mutation bugs in merged preferences
005	hidden_edge_case	bracketed IPv6 host:port parsing
006	type_or_format	decimal-safe price formatting and rounding
007	public_api_preservation	preserve function signature while fixing header behavior
008	regression_trap	keep most-recent unique items without breaking order semantics

Task	Bug type	What it tests
009	minimal_patch	preserve explicit 0 timeout instead of falling through to default
010	dependency_or_config_behavior	backend loading from short names and fully qualified modules

4.2 Prompt conditions

Table 3. Prompt conditions used in the challenge-stage experiments.

Condition	Prompt file	Experimental role
Agent A	agent_a_instruction_only_json.md	Instruction-only repaired-JSON baseline with no examples and no prior patch history.
Agent B	agent_b_final_patch_json.md	Adds a final-success patch pattern, but no failed-attempt history or verifier trajectory.
Agent C	agent_c_vet_json.md	Adds distilled verified-trajectory lessons about failures as evidence, API preservation, and local validation before claiming success.

The only intended experimental change is the conditioning signal provided in the prompt variant. Because these are prompt-level proxies rather than training interventions, condition rankings should be interpreted cautiously.

4.3 Evaluation modes

- Raw diff mode: the model emits a unified diff directly. This mode produced many evaluator-unreachable failures.
- Structured JSON edit mode: the model emits structured file replacement edits; the harness deterministically converts edits into patches.
- Schema repair: shallow schema errors are repaired when possible, and mixed reasoning-plus-JSON outputs are parsed to recover the usable JSON object.
- Repaired JSON replication mode: the final interface used for the five-replication stability study.

4.4 Metrics

We report valid patch rate, verified solve rate, conditional solve rate, false completion rate, schema/status counts, and interface-adjusted metrics. The most important distinction is between patch applicability and semantic success after patch application.

5. Results: From Raw Diffs to Repaired JSON Edits

5.1 Controls and original diff-mode run

Table 4. Original raw-diff run. Agent B and Agent C were heavily confounded by evaluator-unreachable failures.

Set	Tasks	Valid patch rate	Verified solve rate	Solve if applied	False completion
Gold control	10	1.0	1.0	1.0	0.0
Empty control	10	0.0	0.0	n/a	0.0
Agent A	10	0.7	0.5	0.714	0.2
Agent B	10	0.0	0.0	n/a	0.0
Agent C	10	0.3	0.3	1.0	0.0

In the original run, 16 of 22 non-solves were wrapper or backend failures before any patch artifact existed. In the targeted rerun, wrapper failures disappeared from the targeted subset, but 17 of 20 non-solves were still malformed non-empty diffs rejected by git apply. This isolated the remaining bottleneck to patch emission and formatting rather than task design.

5.2 JSON edit mode and schema repair

Table 5. Stage 5 structured JSON edit mode.

Condition	Tasks	JSON ok rate	Patch applied	Verified solve	Solve if applied	False completion
Agent A	10	0.6	0.6	0.5	0.833	0.1
Agent B	10	0.7	0.2	0.1	0.5	0.1
Agent C	10	0.6	0.6	0.5	0.833	0.1

Table 6. Canonical merged Stage 5 + Stage 6 view. Stage 5 and Stage 6 should not be double-counted after this merged table exists.

Condition	JSON original solve	Repaired extra solves	Combined solve rate
Agent A	5/10	0	5/10
Agent B	1/10	3	4/10
Agent C	5/10	1	6/10

Table 7. Interface-adjusted Stage 7 metrics. Each condition has only one semantic failure after patch application.

Condition	Raw tasks	Schema usable	Patch applied	Verified solve	Semantic failure after apply
Agent A	10	9	6	5	1
Agent B	10	10	5	4	1
Agent C	10	9	7	6	1

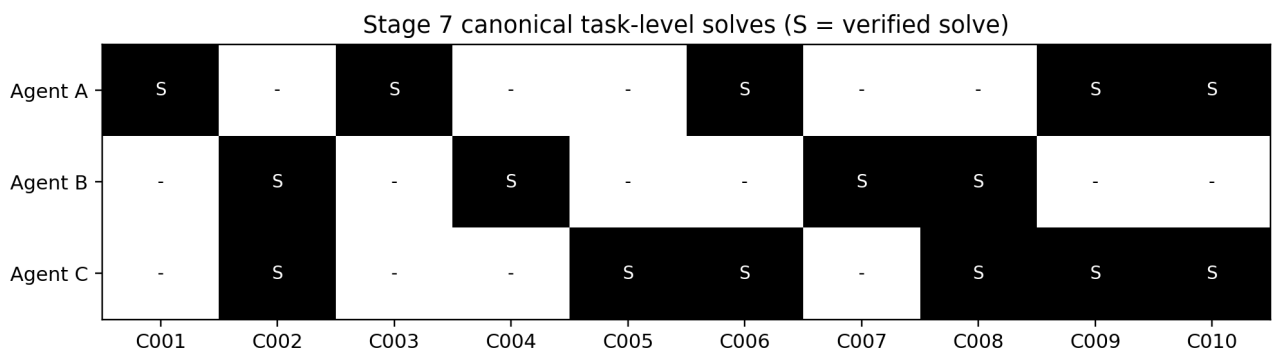

Figure 2. Canonical Stage 7 task-level solve matrix. The table shows that the task set was not impossible: across conditions, every task was solved by at least one condition.

6. Five-Replication Stability Study

Stage 10 repeats the repaired JSON interface across five replications, using the same 10 challenge tasks and the same model/backend pair. Condition order is rotated by replication to reduce ordering effects. The aim is to separate a robust interface claim from a potentially noisy prompt-ranking claim.

Table 8. Stage 10 solve stability across five replications. Intervals are t-intervals over five replication-level solve rates and should be read as pilot uncertainty estimates.

Condition	Mean solve rate	Std dev	Approx. 95% CI	Min	Max
Agent A	0.340	0.230	0.054 to 0.626	0.100	0.700
Agent B	0.360	0.230	0.074 to 0.646	0.100	0.700
Agent C	0.700	0.141	0.525 to 0.875	0.500	0.800

Table 9. Interface reachability and semantic correctness after patch application.

Condition	Mean patch applied	Patch stdev	Approx. 95% CI	Mean solve if applied
Agent A	0.360	0.270	0.025 to 0.695	0.975
Agent B	0.400	0.255	0.083 to 0.717	0.933
Agent C	0.740	0.134	0.574 to 0.906	0.944

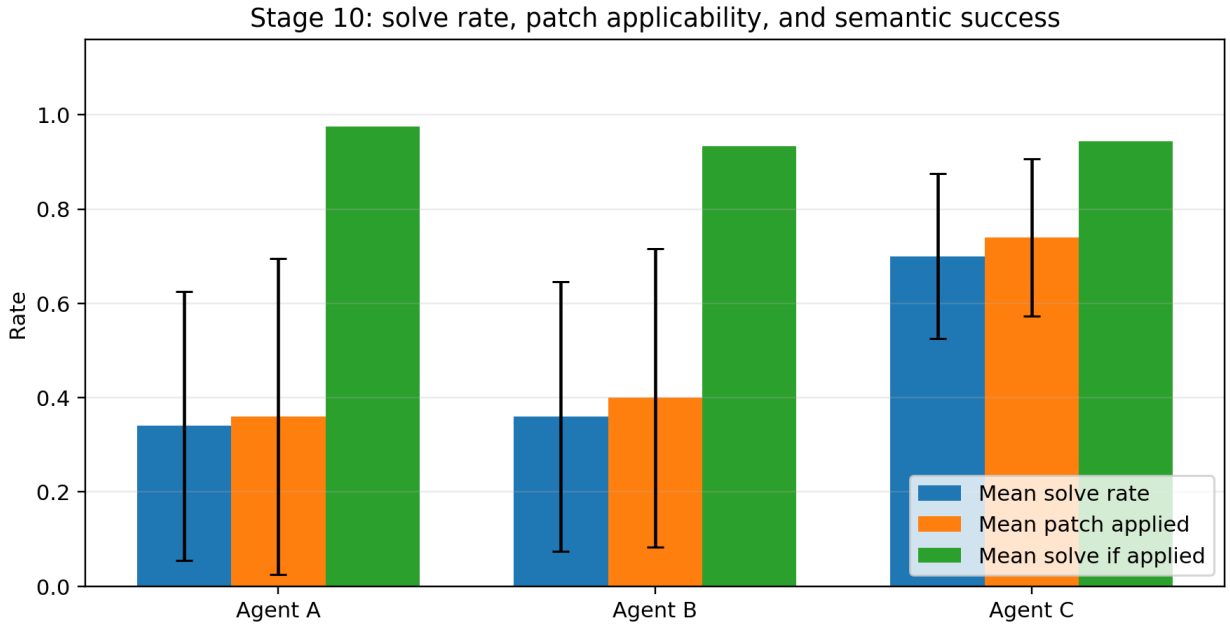

Figure 3. Stage 10 mean solve rate, patch application rate, and solve-if-applied rate. Error bars show approximate t-intervals over five replications for solve and patch application rates.

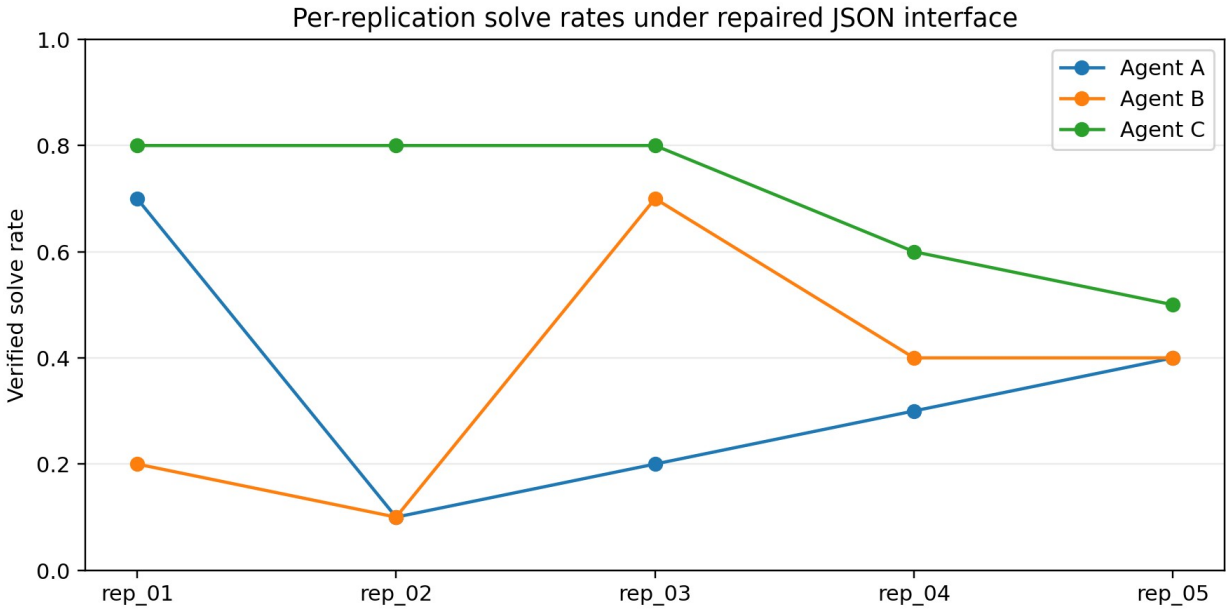

Figure 4. Per-replication solve rates. Agent C is strongest on average, but rankings below Agent C remain noisy.

Table 10. Stage 10 per-replication solve rates.

Replication	Agent A	Agent B	Agent C
rep_01	0.7	0.2	0.8
rep_02	0.1	0.1	0.8
rep_03	0.2	0.7	0.8
rep_04	0.3	0.4	0.6
rep_05	0.4	0.4	0.5

Table 11. Stage 10 aggregated converter status counts across five replications. Here ok is an interface/converter status: the raw output was parsed into a schema-usable edit artifact and a candidate patch could be emitted. It does not imply that git apply succeeded or that tests passed.

Condition	ok	invalid_json	invalid_schema	no_edits	rejected	no_effective_changes
Agent A	41	1	1	6	0	1
Agent B	41	1	3	3	0	2
Agent C	48	0	2	0	0	0

The ok status is intentionally separated from patch-applied and verified-solve metrics. An ok output can still fail later because the generated patch does not apply or because the applied patch does not satisfy the verifier.

Table 12. Stage 10 false-completion counts. False completion denotes cases where an artifact reached evaluation but verification failed despite a success claim or equivalent solve attempt.

Condition	False completions	Rate	Observed in
Agent A	1/50	0.020	rep_01: 1
Agent B	2/50	0.040	rep_05: 2
Agent C	2/50	0.040	rep_02: 1; rep_05: 1

Across five replications, solve-if-applied stays above 0.93 for every condition. This confirms that the repaired interface preserves high semantic correctness once a patch applies. Patch-applied rate tracks solve rate closely, making usable-edit generation the dominant bottleneck in this pilot.

7. Task-Level Stability

The Stage 10 task-level view is reported as a qualitative stability readout rather than as a reconstructed heatmap. Tables 8-12 provide the authoritative aggregate numerical evidence. The task-level readouts show that Agent C is strongest on most tasks, but not uniformly. challenge_007 is the clearest counterexample: Agent A and Agent B are qualitatively stronger on that task, while Agent C consistently fails to reach the semantic evaluation layer. This supports a cautious ranking claim rather than a simplified global superiority claim.

Table 13. Stage 10 qualitative task-level stability readouts. This table is not summed into aggregate totals; Tables 8-12 provide the aggregate numerical evidence.

Task	Bug type	Stage 10 readout
challenge_001	edge_case	Agent C is consistently strongest; Agent A and Agent B usually fail before producing an applicable patch.
challenge_002	multi_file	Agent A is consistently strongest; Agent B and Agent C often fail to reach an applicable patch.
challenge_003	misleading_obvious_fix	Agent C is consistently strongest; Agent B improves over baseline but remains behind.
challenge_004	state_mutation	Agent C is consistently strongest; Agent A consistently fails before the semantic evaluation layer.
challenge_005	hidden_edge_case	Agent C is consistently strongest across replications.
challenge_006	type_or_format	Agent C is consistently strongest; Agent A and Agent B rarely produce a correct applicable

Task	Bug type	Stage 10 readout
		patch.
challenge_007	public_api_preservation	Main counterexample to Agent C dominance: Agent A and Agent B are stronger, while Agent C consistently fails on this task.
challenge_008	regression_trap	Mixed task with only a small Agent C edge; no condition clearly dominates.
challenge_009	minimal_patch	Agent C is consistently strongest across replications.
challenge_010	dependency_or_config_behavior	Agent A and Agent C are roughly comparable; Agent B remains weaker.

8. Discussion

8.1 The robust finding is methodological

The most robust result is not that one prompt condition wins. It is that interface design dominated measured performance. Raw diff generation produced many failures that never reached semantic evaluation. Structured JSON edits, deterministic patch generation, and schema repair made semantic evaluation more reachable and interpretable.

8.2 Prompt rankings are exploratory

Agent C is strongest on average in Stage 10, with the highest mean solve rate and the lowest solve-rate variance. However, the study is underpowered for broad prompt-ranking claims: the benchmark has 10 tasks, one model/backend pair, and five replications over the same tasks. Agent A and Agent B are not reliably separable. Agent C is broad but not universal, as challenge_007 shows. Moreover, Agent C combines trajectory-oriented guidance with more explicit verification-oriented operational instructions, so its advantage should be interpreted as a prompt-level pilot result rather than as isolated evidence for VET training or scaling.

8.3 Why solve-if-applied matters

The high solve-if-applied rates are the core diagnostic finding. Once an edit artifact applied, it usually passed verification. Therefore, a one-number solve rate would mostly measure usable artifact generation in this setting. Benchmarks that do not separate patch applicability from semantic correctness risk misdiagnosing the failure source.

8.4 Implications for verified-experience research

Verified Experience Trajectories remain a useful research direction, but this pilot shows that evaluation infrastructure must come first. If a benchmark cannot distinguish interface failure from semantic failure, then claims about trajectory context, memory, or training are difficult to interpret.

9. Limitations

- This is a pilot study using 10 synthetic software-repair tasks. It is not an external benchmark result.
- Only one model/backend pair was used: gpt-oss:20b-cloud via ollama-cli. Backend-specific behavior may influence results.

- The study does not test VET training or scaling. It tests prompt-level proxies and interface interventions.
- The prompt conditions differ in content and may differ in verbosity, explicitness, and alignment with the JSON edit interface.
- The five replications reuse the same 10 tasks, so variance estimates are useful but not a substitute for a larger independent task set.
- The reported confidence intervals are approximate t-intervals over five replication-level observations. They are not task-distribution confidence intervals, because the same 10 tasks are reused across replications.
- A public release must include exact prompts, model/backend details, decoding parameters, retry policy, raw outputs, and harness scripts for full reproducibility.

10. Future Work

The immediate next step is to extend the task set and rerun the repaired JSON interface on additional models and backends. The broader next step is to test the Verified Experience Hypothesis directly: train or fine-tune agents on verified experience trajectories and compare them with instruction-only and final-output-only baselines under a stable interface. Such a study should vary trajectory quantity, trajectory quality, verifier strength, and memory usage. The present paper establishes a prerequisite: failure decomposition must be in place before those comparisons can be trusted.

11. Conclusion

This pilot supports a narrow but important methodological claim: software-agent evaluation is not trustworthy unless it separates interface failure from semantic task failure. In our experiments, raw diff generation caused many evaluator-unreachable failures. Structured JSON edits, deterministic patch generation, and schema repair improved evaluator reachability. Across five replications, semantic correctness after patch application was high for all conditions, while total solve rate was driven largely by patch applicability. Agent C was strongest on average, but prompt ranking remains exploratory. The strongest conclusion is therefore not that one prompt wins; it is that agent benchmarks must report layered interface and semantic metrics before interpreting solve rates.

References

[1] Zhou, S.; Xu, F. F.; Zhu, H.; et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854, 2023. https://arxiv.org/abs/2307.13854.
[2] Xie, T.; Zhang, D.; Chen, J.; et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. https://arxiv.org/abs/2404.07972.
[3] SWE-bench Verified. Human-validated subset of 500 SWE-bench instances. https://www.swebench.com/verified.html.
[4] Yao, S.; Shinn, N.; Razavi, P.; Narasimhan, K. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.
[5] Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629.
[6] Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023. https://arxiv.org/abs/2302.04761.
[7] Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. https://arxiv.org/abs/2303.11366.
[8] Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; Gonzalez, J. E. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023. https://arxiv.org/abs/2310.08560.
[9] Berkeley RDI. How We Broke Top AI Agent Benchmarks. 2026. https://rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/.

Appendix A. Prompt Templates Used in the Pilot

The prompt templates below are the condition-defining parts of the pilot package. Exact released experiments should include the full concatenated prompt, task text, metadata, model/backend settings, retry policy, and raw model outputs.

System agent rules

```
You are a software engineering agent working inside a sandboxed repository.
Rules:
- Inspect before editing.
- Prefer small, auditable changes.
- Run targeted tests after edits.
- Run broader tests before claiming success.
- Do not modify tests unless the task explicitly requires it.
- Do not claim success unless verifiers pass.
- If a command fails, observe the error and revise your plan.
- Preserve public APIs unless the issue explicitly asks to change them.
- Stop when the repository reaches a verified target state.
```

Agent A: instruction-only condition

```
You will receive a repository and an issue. Fix the issue by editing the code and running tests.
Use the system rules. You do not receive examples of prior successful patches or full trajectories.
```

Agent B: final-patch condition

```
You will receive a repository and an issue. Fix the issue by editing the code and running tests.
You may use examples of final successful patches, but not the failed attempts, observations,
corrections, or verifier history that produced them.
Example final patch pattern:
- Identify the failing behavior.
- Apply the smallest code change that satisfies the issue.
- Run tests.
- Do not modify tests unless explicitly requested.
Use the system rules.
```

Agent C: verified-experience trajectory condition

```
You will receive a repository and an issue. Fix the issue by editing the code and running tests.
You may use the following lessons from verified trajectories:
1. Failed attempts are useful evidence. If a patch causes a new failure, inspect the failure before trying a larger change.
2. Targeted tests should be run after a local edit; broader tests should be run before claiming success.
3. Do not treat visible-test success as full success if the patch changes public API or broad behavior.
4. Revert partial changes when they create unexplained failures.
5. Memory updates should be specific, scoped, source-linked, and not overgeneralized.
Example VET pattern:
- Inspect failing test and stack trace.
- Form a small hypothesis.
- Edit one relevant file.
- Run targeted verifier.
- If verifier fails, record observation, diagnose, and correct.
- Run broader verifier.
- Claim success only after verification.
Use the system rules.
```

Common JSON edit output block

```
Return only valid JSON.
The JSON must have this exact shape:
{
  "claim_solved": true,
  "edits": [
    {
      "path": "src/example.py",
      "content": "complete replacement file content"
    }
  ]
}
Rules:
- Do not output markdown.
- Do not output a git diff.
- Do not include explanations.
- Do not modify tests.
- Only edit files allowed by the task metadata.
- Each "content" value must contain the complete replacement content for that file.
- If you cannot solve the task, return: {"claim_solved": false, "edits": []}.
```

Appendix B. Stage 10 Replication Plan

Table B1. Stage 10 condition order rotation.

Replication	Condition order
rep_01	A -> B -> C
rep_02	B -> C -> A
rep_03	C -> A -> B
rep_04	A -> C -> B
rep_05	B -> A -> C

Appendix C. Canonical Stage 7 Task Matrix

Table C1. Stage 7 canonical task-level outcomes after merging JSON mode and schema repair.

Task	Agent A	Agent B	Agent C
challenge_001	solve	-	-
challenge_002	-	solve	solve
challenge_003	solve	-	-
challenge_004	-	solve	-
challenge_005	-	-	solve
challenge_006	solve	-	solve
challenge_007	-	solve	-
challenge_008	-	solve	solve
challenge_009	solve	-	solve
challenge_010	solve	-	solve

Appendix D. Artifact Manifest

Table D1. Artifact manifest connecting paper tables to generated files.

Root	Primary artifacts
results/challenge	gold_summary.summary.json; empty_summary.summary.json; agent_*_summary.csv; failure_subtypes.csv
results/challenge_rerun	agent_*_summary.summary.json; failure_subtypes.csv; raw and extracted patch artifacts
results/challenge_json	agent_*_summary.csv; agent_* runs; candidates/challenge_json/*/raw, patches, status
results/challenge_json_repaired	agent_*_summary.csv; repaired candidate patches and status files
results/challenge_combined	combined_matrix.csv; interface_adjusted_metrics.csv; canonical agent

Root	Primary artifacts
	summaries
results/challenge_json_replication	replication_manifest.json; agent_*_summary.csv; clean repaired-JSON replication outputs
results/stage10_replications	stage10_summary.csv; stage10_patch_metrics.csv; status_counts.csv; replication_plan.md; stage10_manifest.json
prompts	agent_a_instruction_only_json.md; agent_b_final_patch_json.md; agent_c_vet_json.md
tasks/challenge	challenge_001 through challenge_010 task definitions, metadata, repos, tests, and gold patches

A public release should include raw model outputs, converted patches, converter status files, evaluation JSONs, summary CSVs, task definitions, prompt templates, model/backend version, decoding parameters, retry policy, and scripts needed to regenerate all tables.

Artifact release: repository link forthcoming. The artifact package should contain the files listed above and scripts needed to regenerate all tables.